Table of Contents

Introduction **2**

Create Unit Test Project **3**

Setup the folder structure *3*

Why Mocking is Essential: Isolation *4*

Package Selection: NSubstitute *4*

Unit testing of ItemController.Table(): Positive Test **5**

Setup the Test File Location *5*

Write the Positive Test Code *5*

Execute the Test *8*

Unit Testing of Create() and Debugging: Negative test **9**

Reviewing the Action Method *9*

Write the Negative Test Code *10*

Test Explorer and Testing in Debugging Mode **12**

Demo: Unit Testing

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

Introduction

This demo guides you through writing unit tests for the ItemController class in .NET. It covers creating a unit test project, writing tests for both positive and negative scenarios using **NSubstitute**, and using VS Code's Test Explorer for debugging and inspecting test coverage.

- **Create Unit Test Project:**
  - Add a new xUnit Test Project (MyShop.Tests) side-by-side with your application project (MyShop).
  - Reference the project you want to test (MyShop).
  - Install **NSubstitute** for lightweight, readable dependency mocking and test isolation.
- **Unit Testing of ItemController.Table() – Positive Test**
  - Create a folder for test files, e.g., Controllers, and add a test file ItemControllerTests.cs.
  - Use the [Fact] attribute for the test method.
  - Set up mock instances of IItemRepository and other dependencies.
  - Ensure the ItemController.Table() method is tested in isolation.
  - **Testing Phases:**
    - **Arrange:** Prepare mocks and initial conditions.
    - **Act:** Call the Table() method.
    - **Assert:** Verify the results, ensuring the method returns the correct ViewResult and model.
- **Unit Testing of Create() and Debugging – Negative Test**
  - Test what happens if ItemRepository.Create() fails.
  - Set up the mock to simulate a failure scenario.
- **Test Explorer**
  - Use the Test Explorer in VS Code or the test panel in your IDE to run and review test results.
  - Debugging: Add breakpoints in the TestTable() method and observe “result”.
  - Coverage: see a detailed list of test coverage.

Create Unit Test Project

Before writing tests, we prepare our environment by creating a unit test project and referencing our main application.

Setup the folder structure

1. Verify Your Directory Structure

Open your terminal and make sure you are in the parent directory containing your MyShop application folder using `ls`

ls should show

```
[USER]:Demo-UnitTesting % ls
MyShop
```

Use the following command to add a unit test project with the template xunit:

```
dotnet new xunit -o MyShop.Tests
```

After this, when you list all files and folders, you will see two folders

```
baifanz@Baifans-MacBook-Pro Demo-UnitTesting % ls
MyShop          MyShop.Tests
```

2. Why Keep the Test Project Separate?

Placing MyShop.Tests outside of MyShop is a common software engineering practice for several key reasons:

- **Separation of Concerns:** MyShop contains production application logic; MyShop.Tests contains verification logic.
- **Deployment Exclusion:** Prevents test code, test frameworks, and test data from being compiled into the production build or deployed to production servers.
- **Independent Dependencies:** Allows test-only libraries (like mocking tools) to be installed without bloating or polluting application dependencies.
- **Unambiguous Naming:** Naming the project MyShop.Tests explicitly ties it to MyShop while allowing other projects in larger solutions to have their own dedicated test suites.

In industrial practice, large projects often use the below structure, which is more scalable. For simplicity of our demo, we use the side-by-side MyShop and MyShop.Tests structure.

```
MyShop/                          <-- Root directory
├── MyShop.sln                   <-- Solution file grouping your projects
├── src/
│   └── MyShop/                  <-- Existing MVC project moves here
│       └── MyShop.csproj
└── tests/
    └── MyShop.Tests/            <-- New Unit Test project
        └── MyShop.Tests.csproj
```

3. Link the Test Project to the Main Project

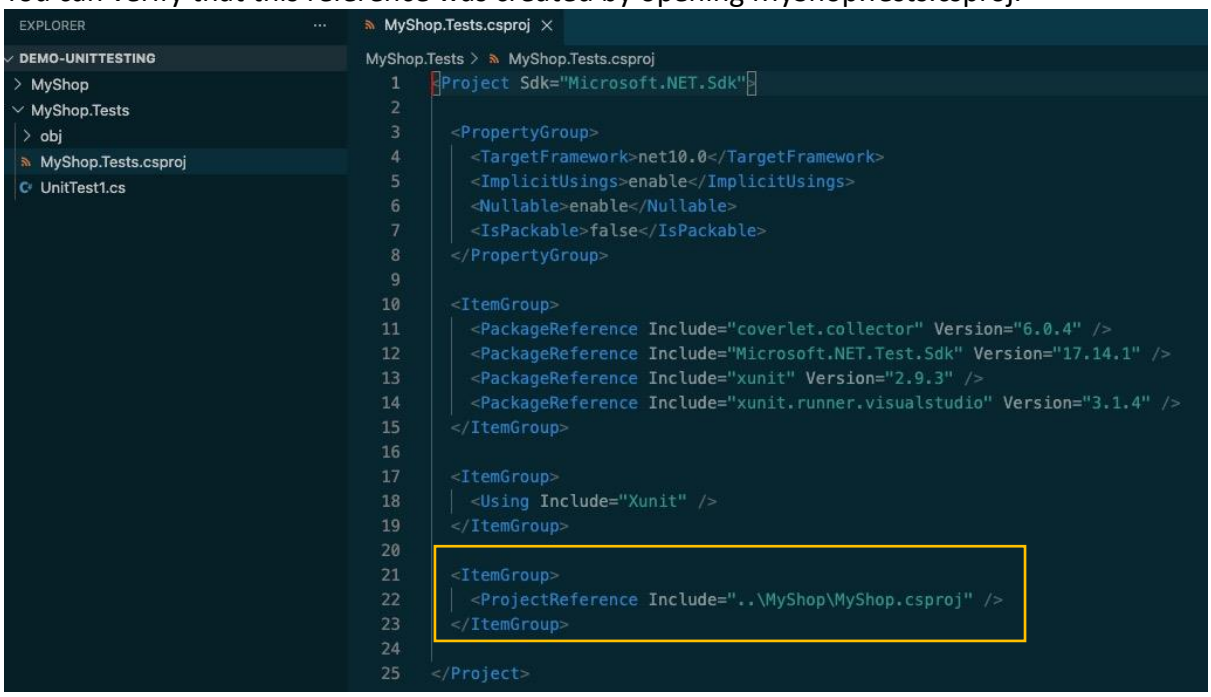
Navigate into the test project folder:

```
cd MyShop.Tests
```

Add a project reference to MyShop. This grants MyShop.Tests access to the controllers, models, and interfaces defined inside MyShop:

```
dotnet add reference ../MyShop/MyShop.csproj
```

You can verify that this reference was created by opening MyShop.Tests.csproj.



4. Install Mocking Package (NSubstitute)
Unit tests must test controllers in isolation without calling real databases or external services. We use **NSubstitute**, a modern, readable mocking library for .NET, to create substitute (mock) implementations of interfaces.
Install **NSubstitute** into the MyShop.Tests project:

```
dotnet add package NSubstitute
```

Why Mocking is Essential: Isolation

A **unit test** must test a single unit of code (such as ItemController) in complete **isolation**. In production, ItemController relies on external dependencies like IItemRepository (which connects to a real database) and ILogger (which writes system logs). If a test uses a real database, it becomes an integration test—it runs slowly, requires database setup, and can fail due to database issues rather than bugs in the controller.

Mocking solves this by creating lightweight "substitute" objects that implement these interfaces. These mock objects simulate real behavior in memory without touching a database, ensuring that any test failure points strictly to the logic inside the controller itself.

Package Selection: NSubstitute

To create mock objects, we use **NSubstitute**, a modern and widely preferred mocking framework for .NET.

- **NSubstitute:** Allows you to substitute real dependencies with mock objects using clean, intuitive C# syntax. It focuses on keeping unit tests readable and isolated with minimal boilerplate code.
- **Historical Context on Moq:** Historically, **Moq** was the standard mocking package in .NET. However, the ecosystem shifted toward alternatives like NSubstitute following a data collection incident in mid-2023.

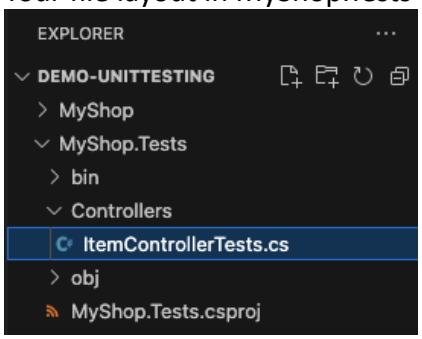
Unit testing of ItemController.Table(): Positive Test

A **unit test** verifies that an individual component (a "unit") of an application works correctly in total isolation from external systems such as databases, web servers, or file systems. In an MVC application, a unit test for a controller action checks if the action returns the expected HTTP result, View, and View Model when given specific inputs and mocked dependency outputs.

Setup the Test File Location

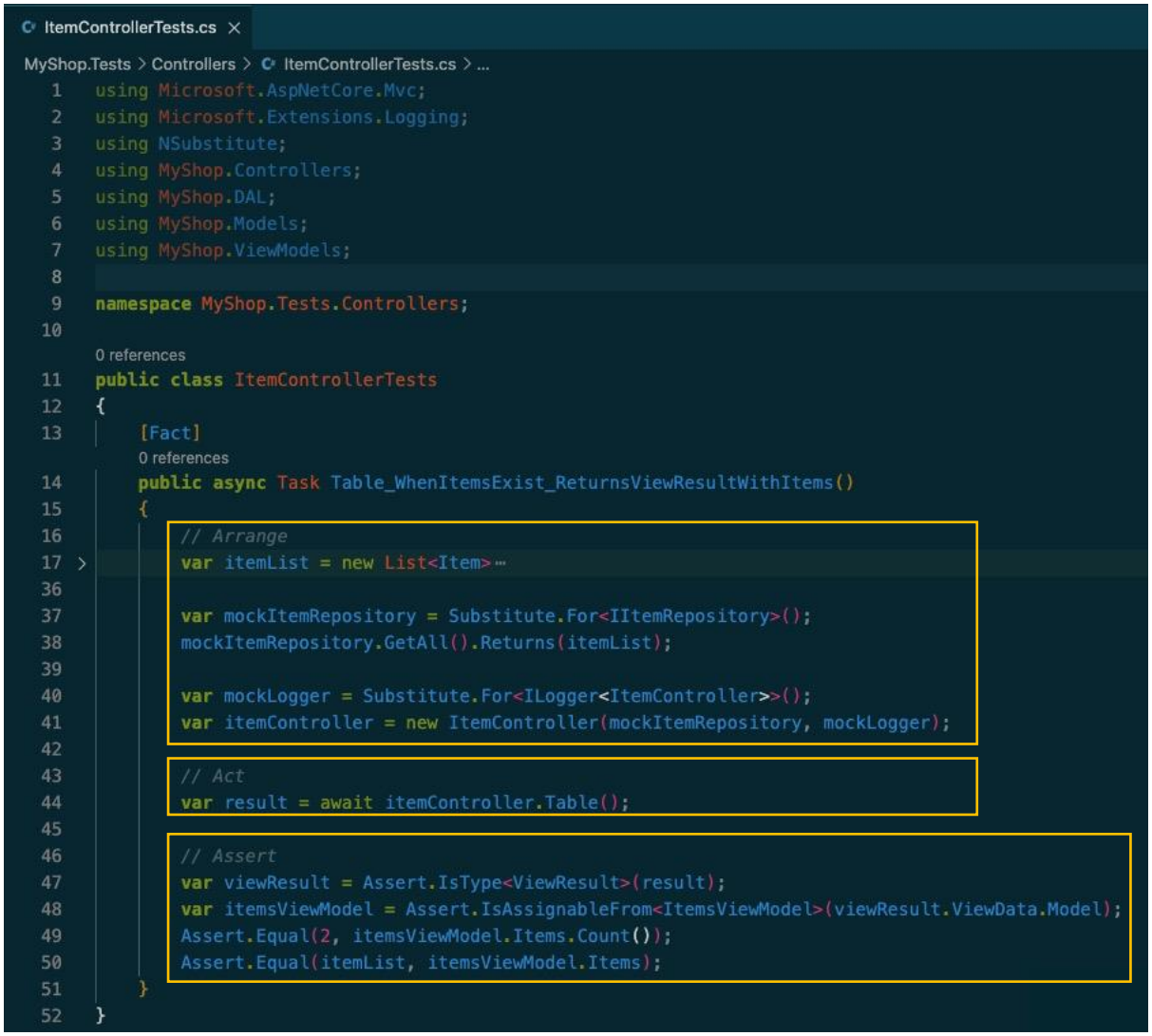
- Inside MyShop.Tests, create a new folder named Controllers.
- Create a new file inside MyShop.Tests/Controllers/ named ItemControllerTests.cs.
- Delete the default template file UnitTest1.cs inside MyShop.Tests/.

Your file layout in MyShop.Tests will look like this:



Write the Positive Test Code

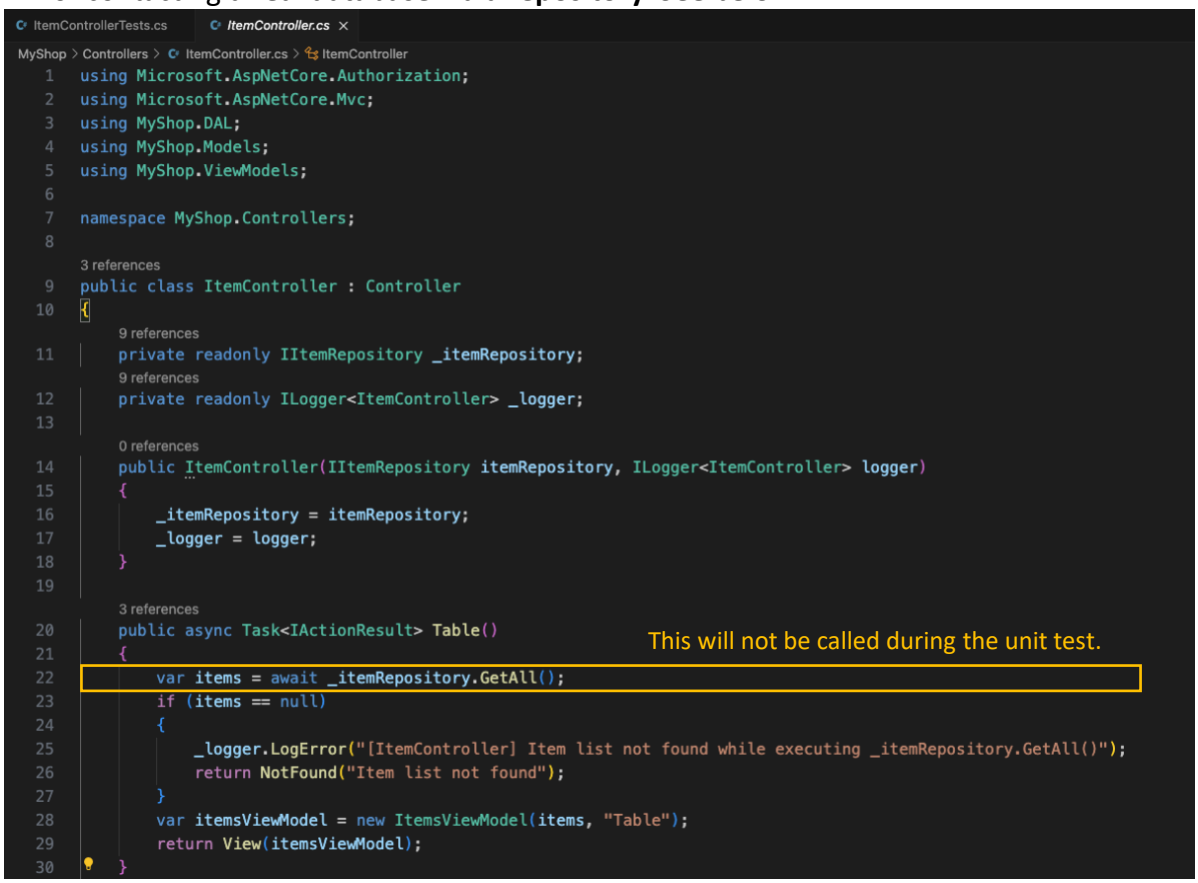
Open ItemControllerTests.cs and add the following code testing the successful execution of Table():



Code explanations:

- **[Fact] Attribute:** In xUnit, test methods must be explicitly marked so the test runner knows to execute them. The `[Fact]` attribute tells xUnit that this method is a standalone, deterministic test that requires no parameters and always tests the same fixed condition.
- **Method Naming (Table_WhenItemsExist_ReturnsViewResultWithItems):** Uses descriptive names with a clear convention: `MethodUnderTest_Scenario_ExpectedResult`.
- **Arrange:**
- **Substitute.For<IItemRepository>():** This creates lightweight mock objects (substitutes) that implement `IItemRepository` and `ILogger<ItemController>` entirely in memory. Because `ItemController` accepts its dependencies through its constructor (Dependency Injection), we can supply these substitutes instead of real database connections or file logging services.
- **mockItemRepository.GetAll().Returns(itemList):** This line is the core mechanism of **isolation**. It configures the substitute repository to listen for calls to `GetAll()`. When `ItemController` calls `_itemRepository.GetAll()` during the test, NSubstitute

intercepts the call and immediately returns the predefined `itemList` in memory, instead of contacting **a real database via a repository. See below**



- **new ItemController(mockItemRepository, mockLogger):** We instantiate the real `ItemController` class, passing our mocked dependencies directly into its constructor. Unlike legacy frameworks like Moq (which required writing `.Object`), NSubstitute substitutes directly inherit the interface type and can be passed as regular arguments.

- **Act:**
- **await itemController.Table():** We invoke the asynchronous `Table()` method directly, storing the returned `IActionResult` in the `result` variable.

- **Assert:**



- `Assert.IsType<ViewResult>(result)` checks that Table() returned a ViewResult (and not NotFound or BadRequest).
- `Assert.IsAssignableFrom<ItemsViewModel>(...)` checks that the model passed to the View is an ItemsViewModel.
- `Assert.Equal(2, itemsViewModel.Items.Count())` verifies the view model contains the expected number of items.

- `Assert.Equal(itemList, itemsViewModel.Items)` checks that the items returned match the exact list defined in the Arrange phase.

The Arrange-Act-Assert (AAA) Pattern

Every well-structured unit test follows three distinct phases:

- **Arrange:** set up the environment for the test. This includes creating any necessary objects, setting initial conditions, and preparing the inputs required for the unit of code you are testing. The goal is to create a controlled environment that mimics the conditions under which the code will be executed.
- **Act:** execute the code or method that you are testing. The purpose of this step is to trigger the behaviour you want to test and obtain the result of the operation.
- **Assert:** check whether the result of the code execution matches your expectations. If the actual outcome aligns with the expected outcome, the test passes. If they don't match, the test fails, and you may need to investigate the cause of the discrepancy.

Mock data:

```
var itemList = new List<Item>
{
    new Item
    {
        ItemId = 1,
        Name = "Fried Chicken Wing",
        Price = 20,
        Description = "Delicious spicy chicken wing",
        ImageUrl = "/images/chickenwing.jpg"
    },
    new Item
    {
        ItemId = 2,
        Name = "Brown Cheese",
        Price = 20,
        Description = "Typical Norwegian cheese",
        ImageUrl = "/images/brunost.jpg"
    }
};
```

Execute the Test

Open your terminal in the MyShop.Tests directory and run:

```
dotnet test
```

Upon completion, you will see output confirming the test passed:

```
[USER]:MyShop.Tests % dotnet test
Restore complete (0,3s)
  MyShop net10.0 succeeded (4,6s) → /Users/bazho0342/Code/baifan-ai-projects/ITPE3200-26H/5-Security-UniTest/Demo-UnitTesting/MyShop/bin/Debug/net10.0/MyShop.dll
  MyShop.Tests net10.0 succeeded (1,2s) → bin/Debug/net10.0/MyShop.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.11)
[xUnit.net 00:00:00.07]   Discovering: MyShop.Tests
[xUnit.net 00:00:00.09]   Discovered:  MyShop.Tests
[xUnit.net 00:00:00.09]   Starting:    MyShop.Tests
[xUnit.net 00:00:00.17]   Finished:    MyShop.Tests
  MyShop.Tests test net10.0 succeeded (1,3s)

Test summary: total: 1, failed: 0, succeeded: 1, skipped: 0, duration: 1,3s
Build succeeded in 7,8s
```

Unit Testing of Create() and Debugging: Negative test

A **negative test** verifies how the application behaves when an operation fails or encounters invalid conditions. While positive tests ensure the application works as intended under normal conditions, negative tests ensure that the software handles errors gracefully without crashing or losing user state.

Reviewing the Action Method

In ItemController.cs, review the HttpPost Create action:

```
55      [HttpGet]
56      [Authorize]
57      public IActionResult Create()
58      {
59          return View();
60      }
61
62      [HttpPost]
63      [Authorize]
64      public async Task<IActionResult> Create(Item item)
65      {
66          if (ModelState.IsValid)
67          {
68              bool returnOk = await _itemRepository.Create(item);
69              if (returnOk)
70                  return RedirectToAction(nameof(Table));
71          }
72          _logger.LogWarning("[ItemController] Item creation failed {@item}", item);
73          return View(item);
74      }
```

This will not be called during the unit test.

- **Positive Outcome:** If _itemRepository.Create(item) succeeds (returns true), the controller redirects to Table.
- **Negative Outcome:** If _itemRepository.Create(item) fails (returns false), the controller logs a warning and redisplays the Create View along with the submitted item model so the user can see their input and try again.

Write the Negative Test Code

Add the following test method to ItemControllerTests.cs inside MyShop.Tests/Controllers/:
We add a test method as below to ItemControllerTests.
Here we want to test what happens when the ItemRepository.Create() fails.
This is a negative test.



```
53      [Fact]
        0 references
54      public async Task Create_WhenCreationFailed_ReturnsViewResultWithItem()
55      {
56          // Arrange
57          var testItem = new Item
58          {
59              ItemId = 1,
60              Name = "Spicy Chicken Wing",
61              Price = 20,
62              Description = "Delicious spicy chicken wing",
63              ImageUrl = "/images/chickenwing.jpg"
64          };
65
66          var mockItemRepository = Substitute.For<IItemRepository>();
67          mockItemRepository.Create(testItem).Returns(false);
68
69          var mockLogger = Substitute.For<ILogger<ItemController>>();
70          var itemController = new ItemController(mockItemRepository, mockLogger);
71
72          // Act
73          var result = await itemController.Create(testItem);
74
75          // Assert
76          var viewResult = Assert.IsType<ViewResult>(result);
77          var viewItem = Assert.IsAssignableFrom<Item>(viewResult.ViewData.Model);
78          Assert.Equal(testItem, viewItem);
79      }
```

Code explanations:

- **Method Naming (Create_WhenCreationFailed_ReturnsViewResultWithItem):**

Clearly identifies the method under test (Create), the scenario (WhenCreationFailed), and the expected outcome (ReturnsViewResultWithItem).

- **Arrange:**
- **Configuring Mock Failure (mockItemRepository.Create(testItem).Returns(false)):**

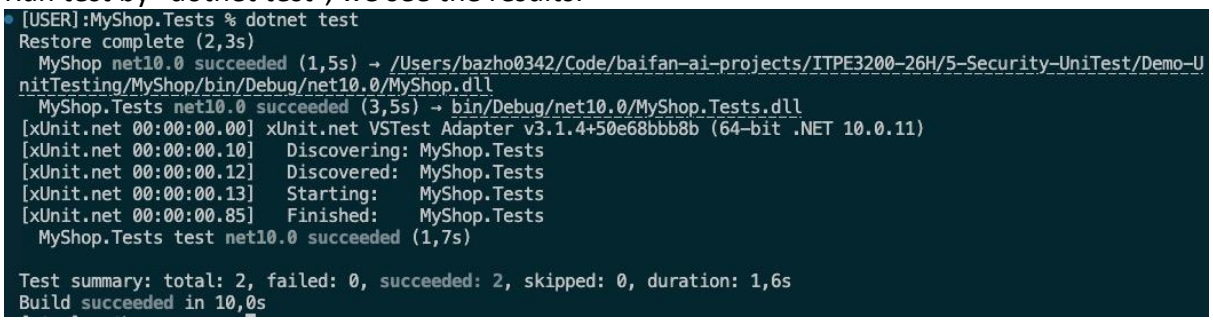
This is the key setup for a negative test. We instruct NSubstitute to simulate a database failure: whenever Create(testItem) is invoked on the substitute repository, it immediately returns false without making any real database calls.

- **Act:**
- **Executing Action Under Failure (await itemController.Create(testItem)):**

Passes testItem into the controller action while the repository is configured to fail.

- **Assert:**
- **Verifying Failure Handling (Assertions):**
    - `Assert.IsType<ViewResult>(result)` verifies that the controller returned a ViewResult (re-rendering the form) rather than a RedirectToActionResult (which would signify success).
    - `Assert.IsAssignableFrom<Item>(viewResult.ViewData.Model)` verifies that the model passed back to the view is of type Item.
    - `Assert.Equal(testItem, viewItem)` ensures that the exact same item object entered by the user is preserved and returned to the view, preventing data loss on the input form.

Run test by “dotnet test”, we see the results:

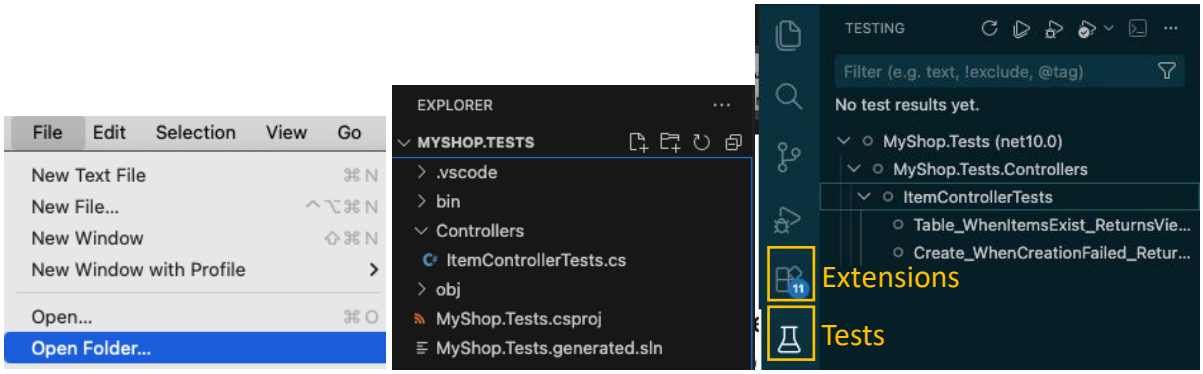


Test Explorer and Testing in Debugging Mode

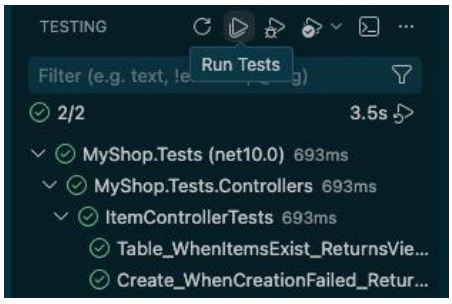
In addition to running tests via the terminal using `dotnet test`, VS Code provides a visual **Test Explorer** interface (powered by the **C# Dev Kit** extension) that allows you to run, debug, and inspect code coverage with a single click.

1. Opening the Test Explorer

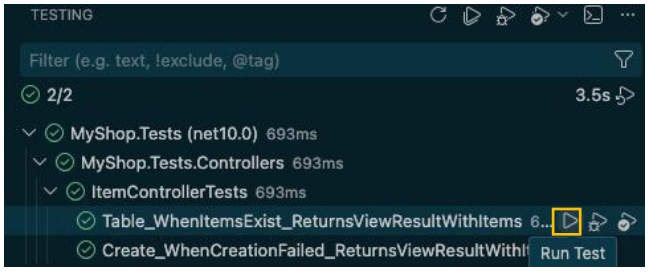
- Open VS Code at the test project folder MyShop.Tests. We need to see that MyShop.Tests is on the top level.
- Click the **Testing** icon (the flask symbol) on the left sidebar. If you do not see the flask symbol, you may need to wait a bit, then the test explorer will recognise the test project.
- **C# Dev Kit** will scan your workspace and build a tree view of your test suite organized by namespace and class name. If you have not installed the C# Dev Kit, you can search and install it via the Extensions tab.



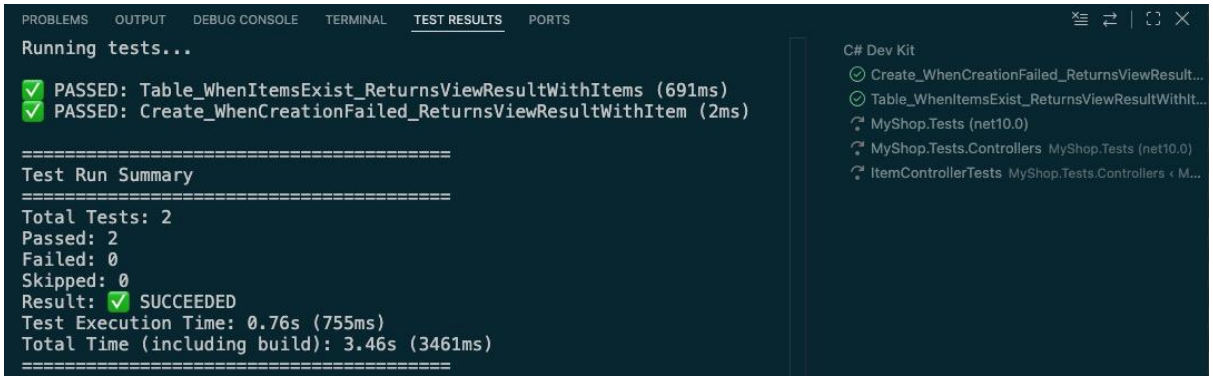
If you don't see it, wait a bit, then the test explorer will recognise the test project.
You can click the run test button at the top, which will run all tests.



You can also click the run test button of each individual test.



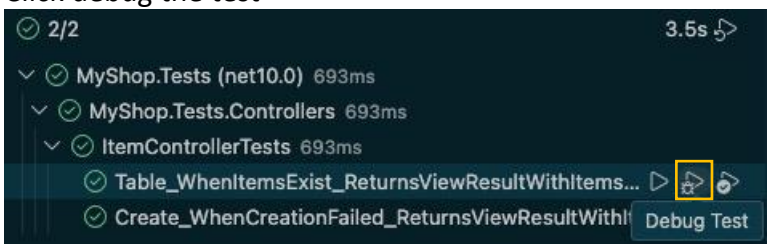
You can click the Test Results panel to see the test results (hook means passed tests):



To further verify how the test is executed, we can add a breakpoint in ItemControllerTests.cs, line 49.

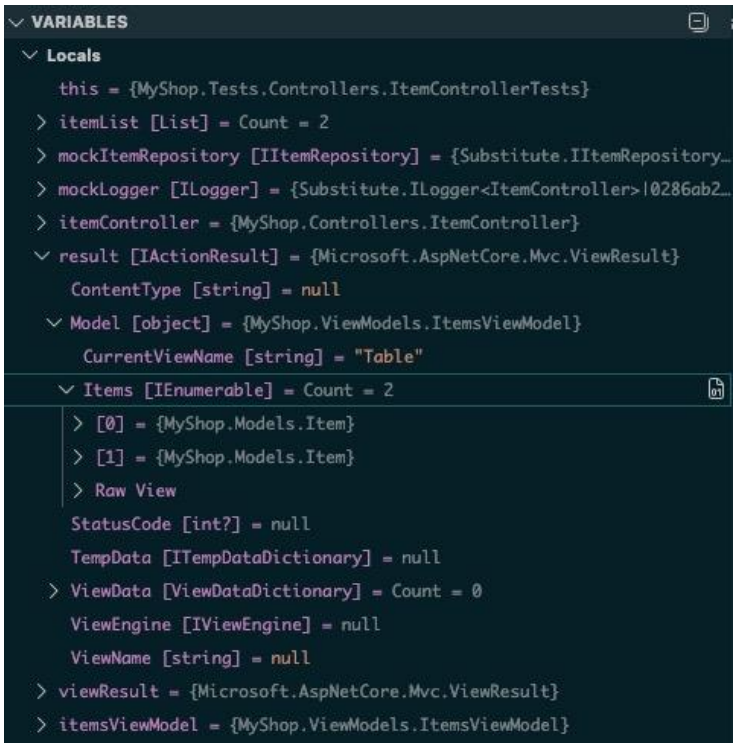


Click debug the test

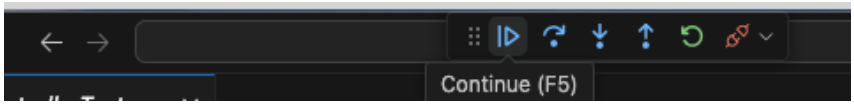


The test will stop at the breakpoint. Go to the variables explorer on the left (next figure), you will see that indeed the result has only two items in its model (which is ItemsViewModel). This means in our test, the repo.GetAll() is indeed replaced by our designed behaviour in Line 38 of ItemControllerTests.cs:

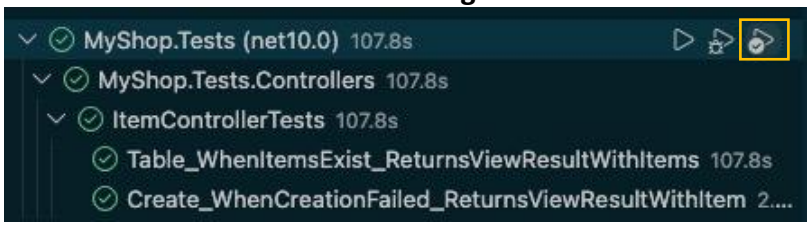
```
mockItemRepository.GetAll().Returns(itemList);
```



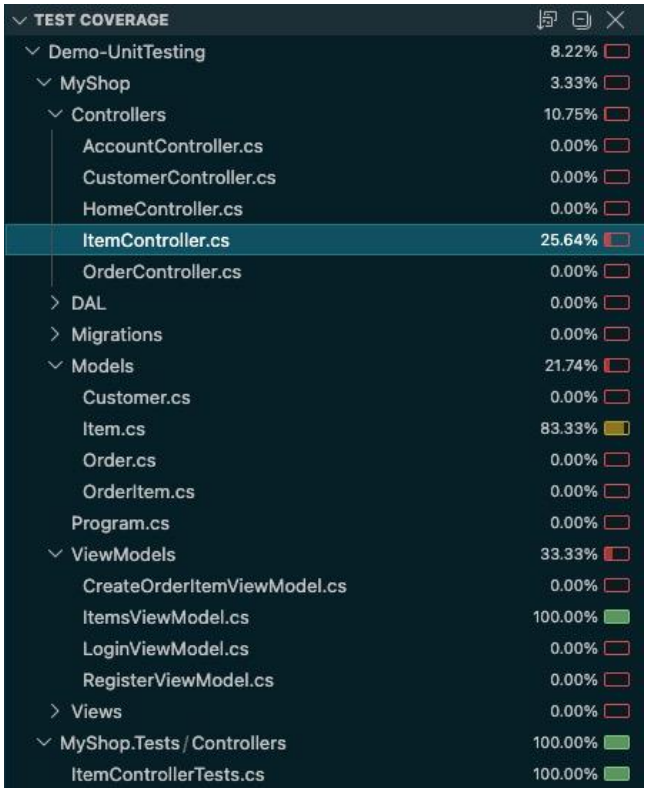
Click the continue button of the debugging panel to let the test continue.



You could also run **test with coverage**



Then in the Test Coverage panel you will get a detailed list of how much code is now covered by the test that you just run:



In addition to **xUnit**, .NET natively supports other popular unit testing frameworks like **NUnit** and **MSTest**. While syntax varies slightly, all of them follow the exact same **Arrange-Act-Assert** pattern and integrate directly into VS Code via the **C# Dev Kit**.

Great job! ☺ You now know how to do unit testing!